

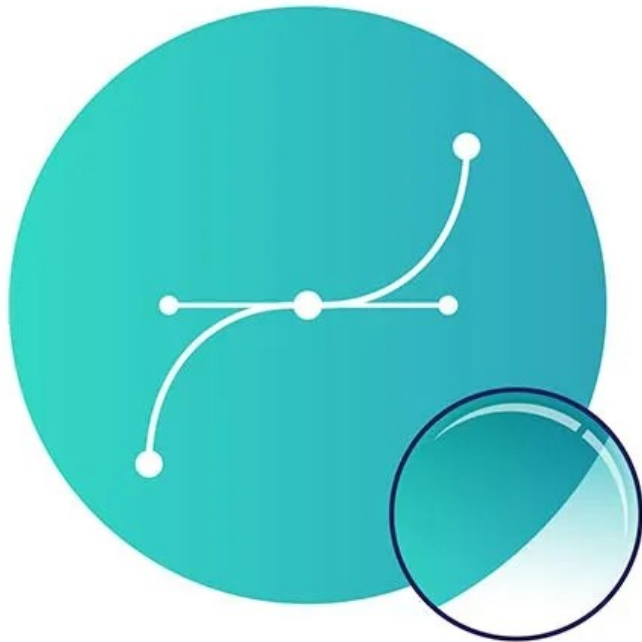
RI204

Windows programiranje

dr.sci. Edin Pjanić

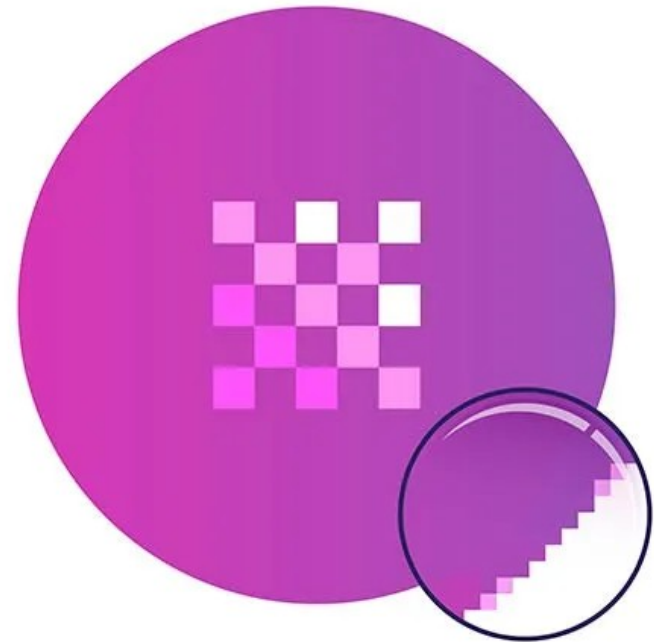
- Grafika nastavak
 - GDI objekti
 - Bitmape
 - Animacije i video igre

- Dva tipa računarske grafike:
 - Vektorska
 - Tretira izlazni uređaj u obliku koordinatnog sistema u kojem su grafički objekti opisani pomoću matematičkih primitiva, tačke, linije, krive itd.
 - Pogodna za tehničke crteže.
 - Bitmapirana (rasterska)
 - Tretira izlazni uređaj kao skup pravougaonika (piksela) malih dimenzija koji mogu biti različito obojeni. Krajnja slika formira se grupisanjem piksela u pravougaonu mrežu.
 - Pogodna za zapis kompleksnih fotografija koje potiču iz realnog svijeta.
- Monitori
 - Bez obzira na tip (LCD ili CRT) prikazuju grafiku u obliku piksela (raster uređaji).

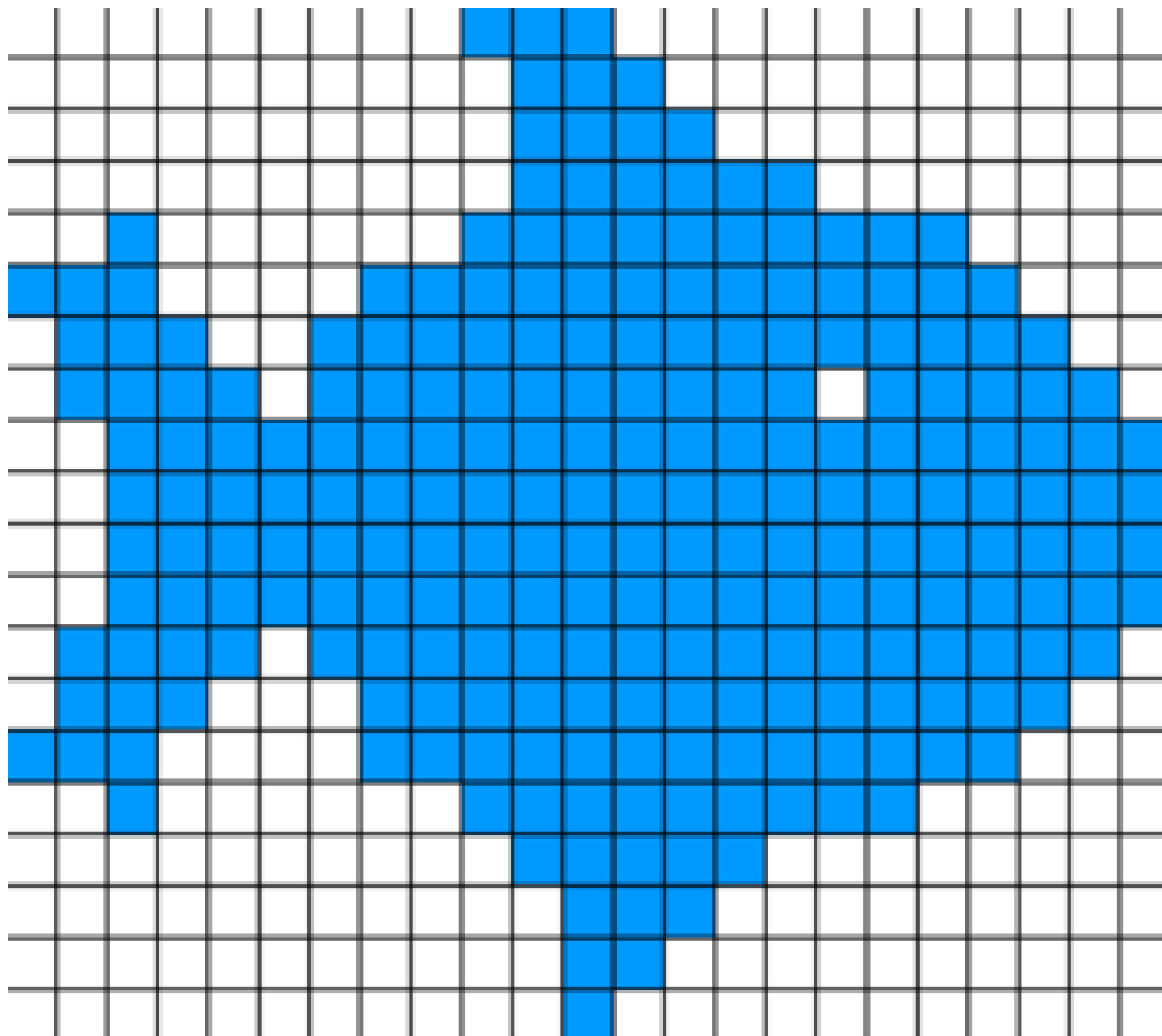
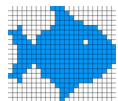


VECTOR

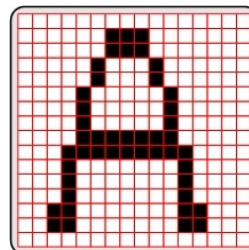
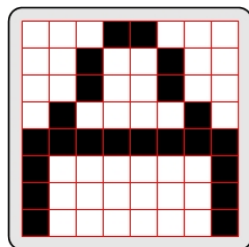
VS



RASTER



- Na slikama su data dva monohromna monitora istih fizičkih dimenzija sa različitim fizičkim rezolucijama
 - Rezolucija se mjeri brojem piksela koje uređaj može prikazati horizontalno ili vertikalno.
 - Veća rezolucija (razlučivost) – prikazuje se više detalja



- Video kartice:
 - Uređaji koji komuniciraju sa monitorom i daju instrukcije za bojenje pojedinačnih piksela na ekranu.
 - Konvertuju digitalnu informaciju (instrukcije o prikazu od strane OS-a) u analognu ili digitalnu formu razumljivu za monitore.

Kodiranje slike

- Bitmapa je uređena sekvenca bajta koja predstavlja pravougaonu mrežu različito bojenih piksela

Hex	Binarno	
0xFF	11111111	← →
0xC3	11000011	
0xDB	11011011	
0xDB	11011011	
0xC3	11000011	
0xDF	11011111	
0xDF	11011111	
0xFF	11111111	

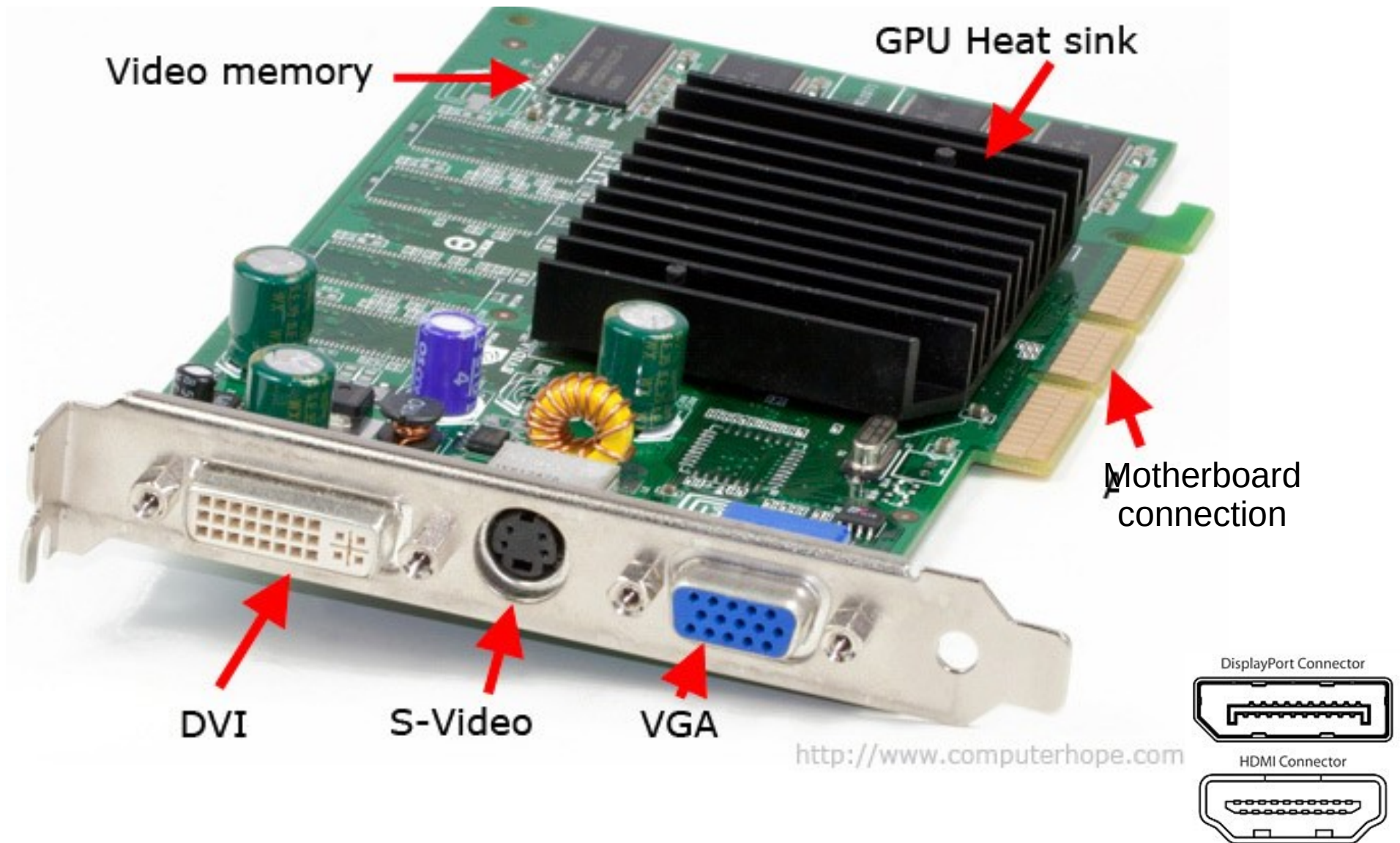
1	1	1	1	1	1	1	1
1	1	0	0	0	0	1	1
1	1	0	1	1	0	1	1
1	1	0	1	1	0	1	1
1	1	0	0	0	0	1	1
1	1	0	1	1	1	1	1
1	1	0	1	1	1	1	1
1	1	1	1	1	1	1	1

- Gornja sekvenca bajta predstavlja bitmapu pod uslovima:
 - Dimenzije 8x8
 - Svaki bajt predstavlja red od 8 piksela
 - Prvi bajt je prvi red a zadnji bajt je zadnji red
 - Najviši bit u bajtu predstavlja prvu kolonu prikaza a najniži predstavlja zadnju kolonu
 - 1 predstavlja bijelu a 0 crnu boju
- Na sličan način se može predstaviti i slika sa više boja. Opisani format je tzv. chunky format; postoje i drugi načini za kodiranje bitmape (planar)

- Video memorija i razlozi za korišćenje:
 - Ako desktop koristi rezoluciju 1024x768 pix, sa bojom 32 bit/pix, to zahtijeva transfer između CPU i video kartice od 3,145,728 byte-a pri svakom ispisu ekrana.
 - Video kartice zbog toga imaju podređenu memoriju (sistemska ili odvojenu), tzv. video memoriju kojoj direktno pristupaju i konvertuju sliku u analogni signal ili u digitalni signal koji razumije monitor.
 - Format kojim se slika kodira unutar video memorije ovisi od hardware-inženjera koji dizajnira video karticu.

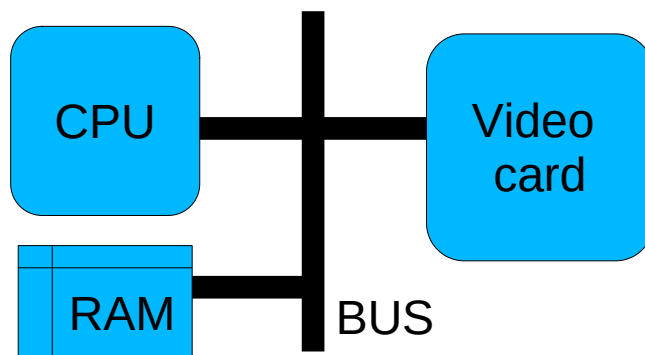
Tipična video karta

9

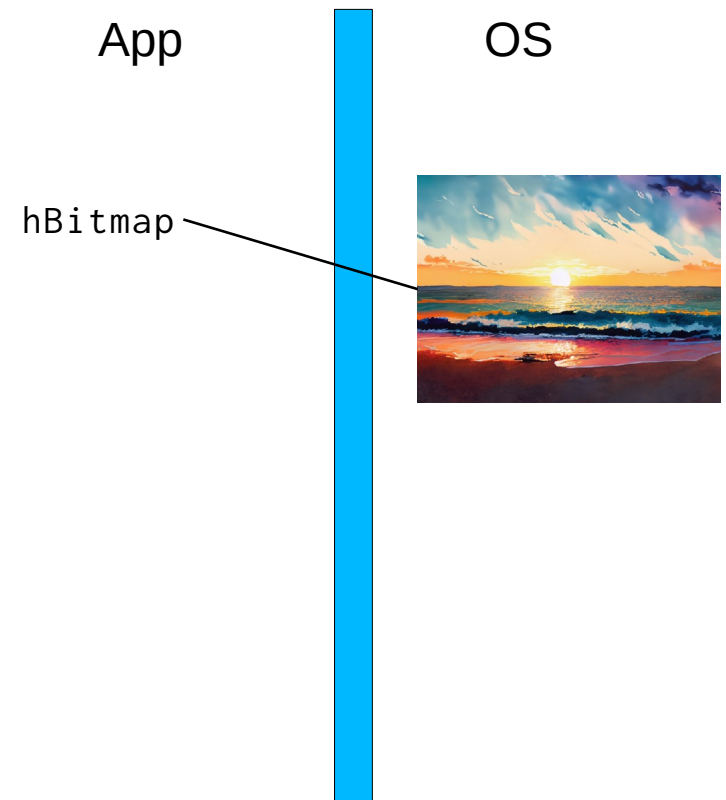


- DDB (Device Dependent Bitmap)
 - Bitmapa kodirana sa specifičnim formatom i brojem boja naziva se DDB jer je u osnovi kreirana za specifičan tip uređaja.
- DIB (Device Independent Bitmap)
 - Bitmapa u generičkom formatu koja se da prilagoditi specifičnom uređaju.
 - Ovaj format je namijenjen za razmjenu bitmape između različitih sistema/hardvera.
 - Windows-i mogu konvertovati DDB u DIB i obrnuto. Takva konverzija uzima CPU resurse.

- “Blit” je operacija kopiranja bitmape iz glavne memorije u video memoriju
 - Moguće pod uslovom da je bitmapa kodirana na identičan način kao video memorija tj. da ima isti format zapisa i istu dubinu boja kao video kartica.
 - Sliku je moguće formirati kao niz bajta koji će se, nakon kopiranja datog niza u video memoriju, na ekranu prikazati kao slika.
 - Blit operacija čini osnovu za 2D animacije i igre i podržana je od strane Windows API-ja.



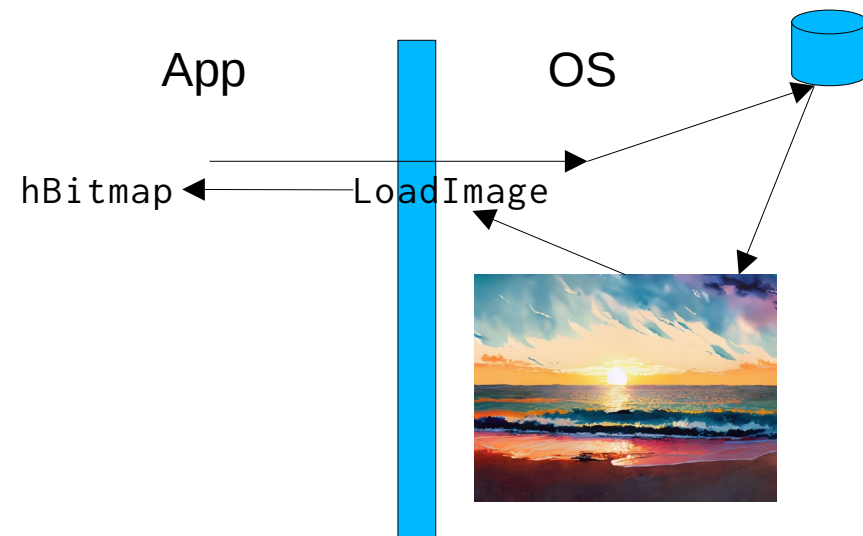
- HBITMAP
 - Windows koristi handle HBITMAP za identifikaciju DDB bitmape.
 - HBITMAP hBitmap;
 - hBitmap je veza sa nekom lokacijom u GDI memoriji na kojoj se nalazi informacija o bitmapi sa specifičnim kodiranjem i dubinom boja.
- Nakon kreiranja varijable tipa HBITMAP potrebno je kreirati bitmapu pomoću neke od API funkcija:
 - Kreiranje nove bitmape u memoriji:
 - CreateBitmap
 - CreateBitmapIndirect
 - CreateCompatibleBitmap
 - Učitavanje postojeće bitmape iz fajla:
 - LoadImage
 - Učitavanje postojeće bitmape iz resursa:
 - LoadBitmap



```
HBITMAP    hbm;  
hbm = (HBITMAP)LoadImage(NULL, "slika1.bmp",  
                           IMAGE_BITMAP, 0, 0, LR_LOADFROMFILE);
```

- Gornja naredba učitava bitmapu iz fajla slika1.bmp
- BMP format je DIB. LoadImage konvertuje DIB u DDB i vraća handle na bitmapu u memoriji
- Nakon korištenja bitmape zauzetu memoriju je potrebno osloboditi isto kao i za ostale GDI objekte pozivom funkcije DeleteObject, npr.

```
DeleteObject(hbm);
```



- Svaki DC ima sa sobom vezanu bitmapu
 - Ova bitmapa je površina po kojoj se vrši crtanje unutar DC-a
 - Kada se vrši crtanje unutar DC-a (npr. sa funkcijama `TextOut` ili `Ellipse`) u osnovi se vrši crtanje po bitmapi koja je vezana za DC i koja se zatim prikazuje direktno na ekranu.
- Zbog toga je nezavisnu bitmapu moguće kreirati i na osnovu kompletne bitmape vezane za DC ili jednog njenog segmenta pomoću funkcije.

```
HBITMAP CreateCompatibleBitmap(HDC hdc, int nWidth, int nHeight);
```

- Gornja funkcija alocira DDB bitmapu kompatibilnu uređaju asociranom sa datim `hdc`-om širine `nWidth` i visine `nHeight`
- Ovako dobijenu bitmapu je na kraju, nakon upotrebe, potrebno dealocirati funkcijom `DeleteObject`

- Uobičajeno DC je vezan za specifičan grafički uređaj, printer ili video. Memorijski DC egzistira samo u memoriji i obično se postavlja da bude kompatibilan sa određenim uređajem a služi kao privremeni bafer.
Kreiranje:
 - `HDC hdcMem = CreateCompatibleDC(hdc);`
 - Gdje je `hdc` handle na postojeći DC, za NULL vrijednost `hdc` kreira se DC kompatibilan video DC-u.
- Nakon upotrebe memorijskog DC-a isti je potrebno uništiti pozivom funkcije
 - `DeleteDC(hdcMem);`
- Bitmapa inicijalno vezana za memorijski DC dimenzija je 1x1 i monohromna.
- Za crtanje unutar ovakvog DC potrebno je selektirati korisnu bitmapu, kompatibilnu sa DC-om na osnovu kojeg je memorijski DC kreiran. Selektiranje bitmape u DC se vrši pozivom funkcije `SelectObject`:
 - `SelectObject(hdcMem, hBitmap);`
- Po bitmapi memorijskog uređaja se dalje može crtati identično kao po bitmapi vezanoj za klasičan DC, tj. DC za stvarni uređaj.

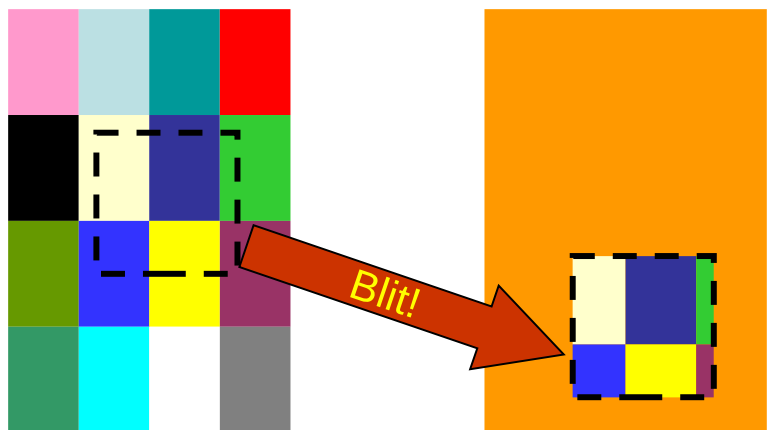
- Bitmapu učitano iz fajla nije moguće direktno selektirati (postaviti) u DC nekog uređaja već je potrebno:
 - Kreirati memorijski DC kompatibilan sa videom
 - Selektirati bitmapu u memorijski DC
 - Izvršiti blit operaciju iz memorijskog DC u DC stvarnog uređaja
 - Selektirati originalnu bitmapu u memorijski DC
 - Uništiti memorijski DC
- Dobijanje dimenzija učitane bitmape - `GetObject` (potrebno za blit

```
HBITMAP    hbm;  
BITMAP     bitmap;  
hbm = (HBITMAP)LoadImage(NULL, "slika1.bmp",  
                           IMAGE_BITMAP, 0, 0, LR_LOADFROMFILE);  
GetObject (hbm, sizeof (BITMAP), &bitmap);
```

- Windows OS popunjava strukturu `bitmap` sa informacijama o učitanoj bitmapi. Širina je u polju `bitmap.bmWidth` a visina u polju `bitmap.bmHeight`

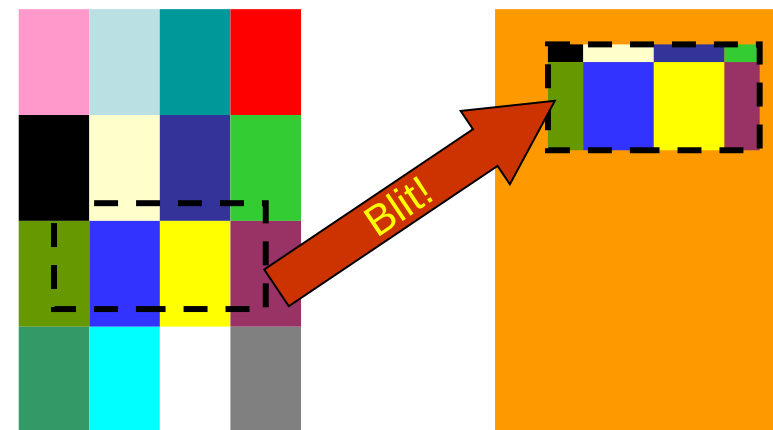

```
BOOL BitBlt(HDC      hdcDest,  // destinacijski DC
            int       xDest,    // x gornji lijevi ugao dest. DC
            int       yDest,    // y gornji lijevi ugao dest. DC
            int       cx,      // širina blitanog pravougaonika
            int       cy,      // visina blitanog pravougaonika
            HDC       hdcSrc,   // izvorni DC
            int       xSrc,     // x gornji lijevi ugao izv. DC
            int       ySrc,     // y gornji lijevi ugao izv. DC
            DWORD     dwROP);   // raster operacija (SRCCOPY
                                // za osnovni tip blit operacije)
```

- Blit operacija se obavlja iz DC u DC i vrši se između kompatibilnih DC-a. dwROP modificira način na koji se kombinuju izvorni i destinacijski pikseli.



izvorni DC

destinacijski DC

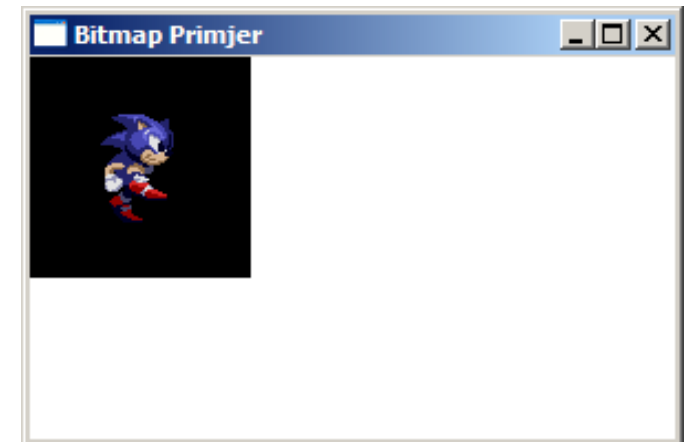


izvorni DC

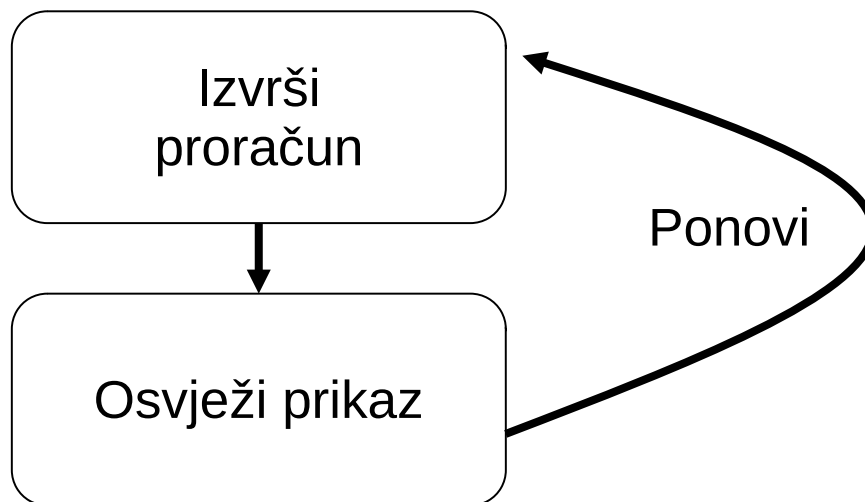
destinacijski DC

```
LRESULT CALLBACK WndProc (HWND hwnd, UINT message, WPARAM wParam, LPARAM lParam)
{
    HDC          hdc, hdcMem;
    HBITMAP      hbmpSlika, hbmpMemStari;
    PAINTSTRUCT  ps;
    BITMAP       bitmap;

    switch (message)
    {
        case WM_PAINT:
            hdc = BeginPaint(hwnd, &ps);
            hbmpSlika = (HBITMAP)LoadImage(NULL, "slika1.bmp", IMAGE_BITMAP, 0, 0, LR_LOADFROMFILE);
            GetObject(hbmpSlika, sizeof(BITMAP), &bitmap);
            hdcMem = CreateCompatibleDC(hdc);
            hbmpMemStari = (HBITMAP)SelectObject(hdcMem, hbmpSlika);
            BitBlt(hdc, 0, 0, bitmap.bmWidth, bitmap.bmHeight, hdcMem, 0, 0, SRCCOPY);
            SelectObject(hdcMem, hbmpMemStari);
            DeleteDC(hdcMem);
            DeleteObject(hbmpSlika);
            EndPaint(hwnd, &ps);
            return 0;
        case WM_DESTROY:
            PostQuitMessage (0) ;
            return 0 ;
    }
    return DefWindowProc (hwnd, message, wParam, lParam) ;
}
```



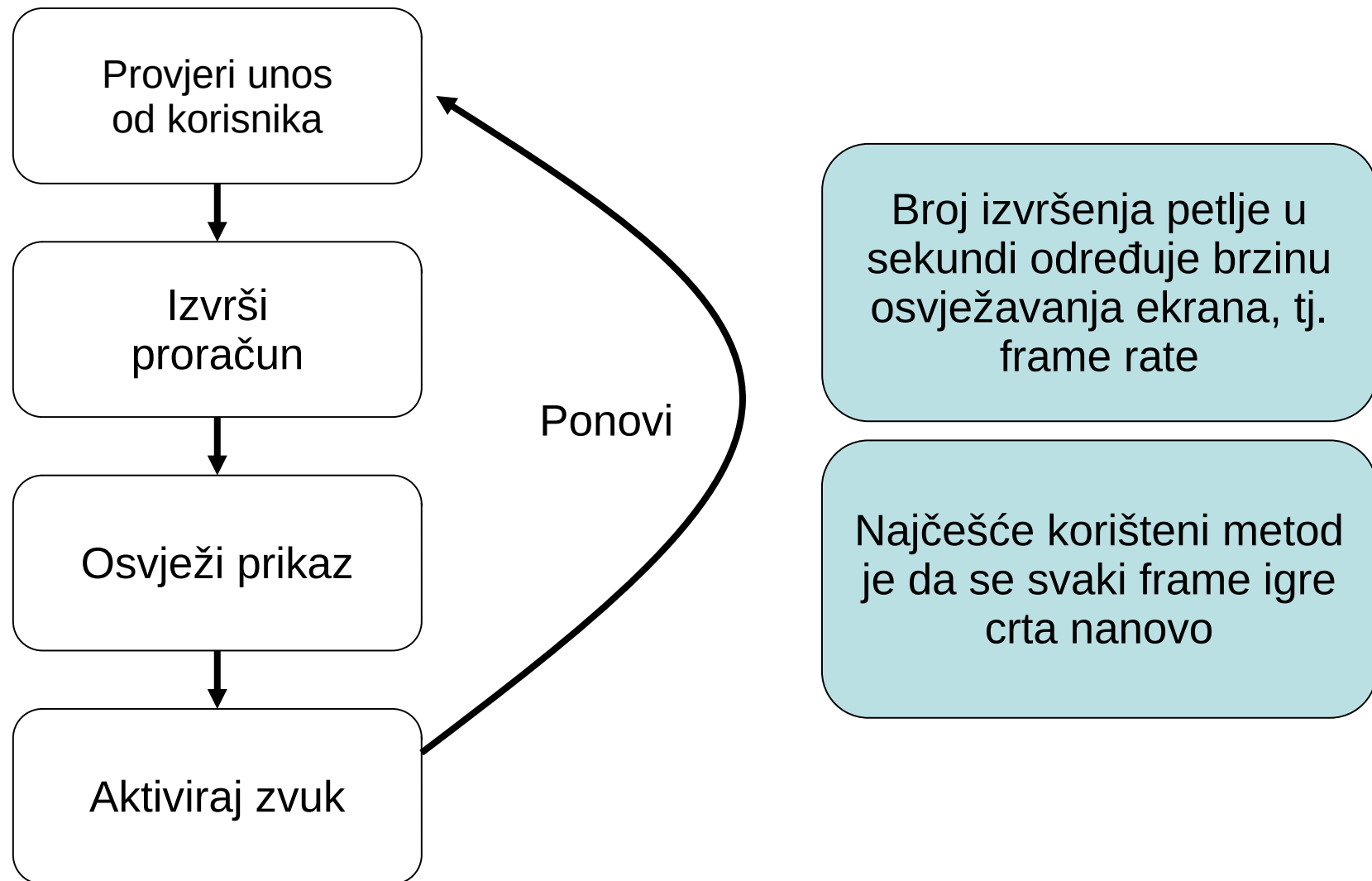
- Igra
 - ... je interaktivni film koji se kreira na način da se serija sukcesivnih slika odgovarajućom brzinom smjenjuje na ekranu.
 - Zadatak programera je da napiše kod koji odlučuje koji elementi se prikazuju u slikama i na kojim lokacijama u određeno vrijeme.
- Za pisanje ozbiljnih igara potrebno je poznavati:
 - Tehnike za programiranje grafike i ulaza
 - Matematika i fizika (krucijalno)
 - Linearna algebra i numeričke metode
 - Simulacija dinamičkih sistema
 - Strukture i algoritme
 - OOP tehnike
 - Implementacije su obično u C++



Broj izvršenja petlje u sekundi određuje brzinu osvježavanja ekrana, tj. frame rate

Kostur igre (interaktivne animacije)

21



- Standardna petlja u WinMain-u

```
while (GetMessage (&msg, NULL, 0, 0))  
{  
    TranslateMessage (&msg) ;  
    DispatchMessage (&msg) ;  
}
```

- Petlja sa stanovišta igre stvara problem:

- GetMessage blokira: ukoliko u redu poruka za prozor nema poruka program prelazi u stanje čekanja dok ne dođe naredna poruka.
- Cilj kod igara je što prije osvježiti ekran.

- Moguće rješenje:

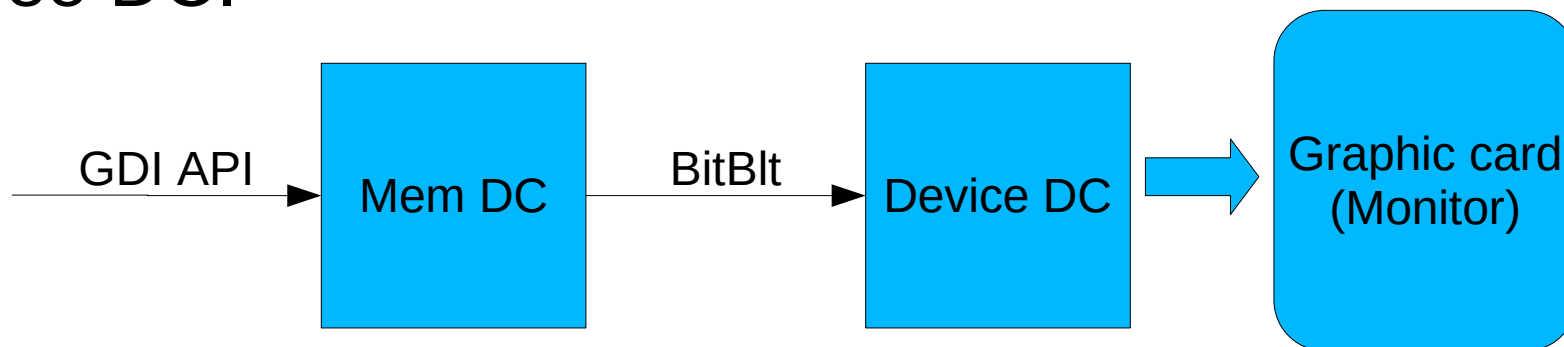
- Napraviti timer koji u određenim intervalima pozivom funkcije `InvalidateRectangle` čini nevalidnim kompletno klijent područje prozora (nezadovoljavajuće).

- Modifikovati petlju na sljedeći način:

```
while (1)
{
    if (PeekMessage(&msg, NULL, 0, 0, PM_REMOVE))
    {
        if (msg.message == WM_QUIT)
            break;
        TranslateMessage (&msg) ;
        DispatchMessage(&msg);
    }
    ProvjeriUlaze();
    IzvrsiRacunanje();
    OsvjeziEkran();
    OsvjeziZvuk();
}
```

- Funkcija PeekMessage ne blokira već:
 - Vraća nenultu vrijednost ukoliko postoji nova poruka u redu poruka.
 - Vraća nultu vrijednost ako nema poruke u redu poruka
 - Zadnji parametar reguliše da li PeekMessage uklanja poruke iz reda.
- Ako nema novih poruka iscrtava se naredni frame.
- Ako postoji poruka, frame će se iscrtati nakon obrade poruke.

- Ukoliko se vrši crtanje direktno na video DC često nastaje problem treperenja (flicker) koji se izbjegava pomoću double buffering-a.
- Flicker nastaje ukoliko video kartica počne da šalje signale za prikaz na ekranu u trenutku dok se još uvijek generiše frame (nepotpuno iscrtan).
- Double buffering podrazumijeva ispis trenutnog frame-a igre najprije na memorijski DC a zatim kopiranje (blit) na video DC.



- Procedura korištenja ove tehnike se vrši u sljedećim koracima:
 - Dobiti video DC (GetDC ili BeginPaint)
 - Na osnovu dobijenog DC kreirati memorijski DC (CreateCompatibleDC).
 - Kreirati bitmapu na osnovu bitmape u DC-u uređaja (CreateCompatibleBitmap).
 - Selektovati bitmapu u memorijski DC (SelectObject).
 - Izvršiti crtanje kompletnog frame-a igre u memorijski DC:
 - funkcije za crtanje grafike i teksta
 - funkcije za rad sa bitmapama
 - Izvršiti blit operaciju iz memorijskog u video DC (BitBlt).
 - Vratiti staru bitmapu u memorijski DC (SelectObject).
 - Izbrisati kreiranu bitmapu i memorijski DC (DeleteObject i DeleteDC).
 - Otpustiti video DC (EndPaint ili ReleaseDC).